Quantstamp

Greek.Fi

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	DeFi: Options
Timeline	2026-06-15 through 2026-06-18
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Source Code	• greekfi/greekfi #4e92aa4 • greekfi/clone #ec7c0a8
Auditors	• Paul Clemson Auditing Engineer • Andy Lin Senior Auditing Engineer • Andrei Stefan Auditing Engineer

Documentation quality	Medium
Test quality	High
Total Findings	10 Fixed: 7 Acknowledged: 3
High severity findings	0
Medium severity findings	0
Low severity findings	6 Fixed: 4 Acknowledged: 2
Undetermined severity findings	0
Informational findings	4 Fixed: 3 Acknowledged: 1

Summary of Findings

Greek.fi is an on-chain options protocol for creating fully collateralized ERC20 option markets. Each market consists of a paired long-side `Option` token and short-side `Receipt` token. Users mint options by depositing collateral, receiving both the long option and the corresponding receipt. Long holders may exercise during the valid exercise window by paying the consideration asset and receiving collateral, while receipt holders represent the short side and can redeem remaining collateral or consideration according to post-exercise settlement rules.

Option behavior is time-gated and supports both American-style options, exercisable before expiry and through the exercise window, and European-style options, exercisable only after expiry and before the exercise deadline. The system also includes factory-level allowances, operator approvals, delegated exercise/redeem permissions, and optional auto-mint/auto-burn behavior for users who want transfers to automatically mint missing option balances or burn matched option/receipt positions.

The codebase has a strong test suite with a mixture of unit, fuzz and stateful invariant tests that cover the protocol's expected behaviour as well as potential edge cases.

During the audit, no high or medium severity issues were discovered. However, a number of low and informational severity issues were highlighted, as well as some auditor suggestions. It is recommended that the client review each of these carefully and decide whether to implement the recommended changes prior to deployment.

Fix-Review Update 2026-06-24:
During the fix review period the client fixed or reasonably acknowledged the report's findings.

Additionally, during the fix review period the decision was made to migrate the protocol's smart contracts out of the protocol monorepo and into its own standalone `greekfi` repository. Due to this the commits listed for each fix and the test suite referred to in the "Test Suite Results" and "Code Coverage" sections of the report may not be present in the new repository.

Fix-Review Update 2026-07-28:
Quantstamp reviewed an additional change to the codebase which gave users the power to grant action specific permissions to approved operators, these actions are `TRANSFER`, `MINT`, `BURN`, `REDEEM`, and `EXERCISE`. Previously, granting an operator approval automatically gave them the power to handle all of these actions. No issues were found to be caused by this feature addition.

ID	DESCRIPTION	SEVERITY	STATUS
GRK-1	**Batch `exerciseFor()` Exercises Live Full Balances Without Caller Spend Bounds**	• Low	Fixed

ID	DESCRIPTION	SEVERITY	STATUS
GRK-2	**Approved Operators Have Significant Spending Power**	Low	Acknowledged
GRK-3	**Unbounded `strike` Can Overflow `Receipt.toConsideration()` and Block Settlement**	Low	Fixed
GRK-4	**Non-Standard Erc-20 Behaviours Such as Fee-on-Transfer and Rebasing Tokens Are Not Supported by the Protocol**	Low	Fixed
GRK-5	**Dust-Sized `Receipt._redeem()` Calls Can Burn Receipts for Zero Payout**	Low	Acknowledged
GRK-6	**Lifetime Counters Can Saturate and Permanently Disable `Option.exercise()`**	Low	Fixed
GRK-7	**`Receipt.redeem()` Does Not Explicitly Guard Pre-Expiry European Redemption Attempts**	Informational	Fixed
GRK-8	**`sweep()` Remains Unavailable While Any Receipt Dust Remains Outstanding**	Informational	Acknowledged
GRK-9	**Late `Receipt.exercise()` State Update Breaks Checks-Effects-Interactions Ordering**	Informational	Fixed
GRK-10	**Direct `OptionUtils.details()` Calls Can Return the Wrong Option Address**	Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:

1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.

3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The following files were considered in scope for the audit:

Files Included

- greekfi/greekfi/contracts/Factory.sol
- greekfi/greekfi/contracts/Option.sol
- greekfi/greekfi/contracts/OptionUtils.sol
- greekfi/greekfi/contracts/Receipt.sol
- greekfi/clone/src/Clone.sol
- greekfi/clone/src/ClonesWithImmutableArgs.sol

Repo: `https://github.com/greekfi/clone`

Repo: `https://github.com/greekfi/greekfi`

Operational Considerations

- **Receipt redemption is first-come, first-served across consideration and collateral pools:** Receipt redemption pays from the consideration pool first and then from the collateral pool when collateral redemption is available, so short-side holders may receive different token mixes depending on prior exercise and redemption activity. This includes the ability for users to mint receipt tokens just-in-time, and call `redeem()` instantly, ahead of users who held positions when the Option was exercised. User-facing materials and interfaces should document this behavior and avoid presenting receipt redemption as fully fungible across time when pool composition can affect the token received.
- **Auto-mint can create unwanted collateralised positions:** When `autoMintBurn` is enabled, transfers above the sender's current option balance can mint the deficit by pulling collateral through the factory allowance registry instead of reverting for insufficient balance. Interfaces should treat the combination of `autoMintBurn`, `approveOperator`, token approval, and factory allowance as minting authority over the user's collateral and warn users before enabling it.
- **Option tokens become untransferable after `exerciseDeadline`:** Alone, this does not present a significant security risk, as an unexercised option after the deadline has no inherent value. However, any external protocol that interacts with the Option ERC-20 tokens must be aware that the transfer of Option tokens is not possible after the `exerciseDeadline`.
- `allowExercise` **is, by design, a withdrawal right over ITM value:** `Option.exerciseFor(holder, amount)` routes the released collateral to `msg.sender` rather than the holder, so an authorised keeper can exercise the holder's in-the-money option and keep the intrinsic value. The code documents this as the "dangerous keeper path."
- **Constrain delegated operator workflows:** Keep `approveOperator`, max collateral allowances, and `autoMintBurn` behind audited allowlists, explicit confirmations, and clear revocation runbooks. UI and integration flows should distinguish protocol-wide max approvals from per-market approvals and strongly gate free-form operator approval.
- **Validate market-maker quote inputs before signing:** The market maker should sign quotes only after confirming that quote math matches the on-chain token denomination, actual consideration decimals, put normalization, explicit per-token price feeds, fresh pricing inputs, and the configured policy for American options and post-expiry exercise windows. Unsupported or stale markets should be declined rather than priced.
- **Gate production quote signing:** Public or externally reachable quote surfaces should not expose the production maker signing key without caller controls. If public signed quotes are required, add authenticated taker access, per-taker and global notional caps, rate limits, tight CORS, short expiries, maker-generated nonces, nonce tracking until expiry, monitoring, and a key-rotation runbook.
- **Treat option discovery as a canonicality-sensitive process:** Market-maker and integration indexing should use the compiled Factory ABI, wait for confirmations, rescan a bounded overlap window, store block hashes, prune non-canonical logs, and verify discovered option metadata on-chain before enabling quotes.

Key Actors And Their Capabilities

The protocol has the following authorized roles:

Factory Owner

- The factory owner is capable of sweeping leftover tokens from `Receipt` instances when there are no outstanding Receipt tokens to be redeemed.

User-Approved Operators

- User-approved operators can mint Option/Receipt pairs on the user's behalf.
- User-approved operators can transfer Option tokens on the user's behalf.

User-Allowed Exercisers

- User-allowed exercisers can exercise a user's Option tokens by providing the required consideration tokens. It is important to note that the collateral gained is transferred to the keeper rather than the user.

User-Allowed Redeemers

- User-allowed redeemers can redeem a user's Receipt tokens on their behalf.

Findings

GRK-1

Batch `exerciseFor()` Exercises Live Full Balances Without Caller Spend Bounds

Low Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `3bc63217efabe4bb922d56b29a2af5e2e2f1dc02`, `f18eb3132d19db1e929fb71345e5c972e3cbac12`, and `716e9ffde9ea30b130a0ca2fb9024a91a81d7064`.

File(s) affected: `foundry/contracts/Option.sol`

Description: `Option.exerciseFor(address[] calldata holders)` allows a caller to batch exercise the full option balance of each authorized holder. For each holder in the input array, the function reads the holder's live option balance with `balanceOf(h)`, skips zero balances and unauthorized holders, then burns the full live balance and calls `receipt.exercise(msg.sender, a)`. `Receipt.exercise()` pulls the required consideration from `msg.sender` through the factory allowance registry and sends the exercised collateral to `msg.sender`, so the batch caller pays the strike consideration for every holder exercised by the batch.

The batch function does not check per-holder maximum amounts, or a total consideration spend cap. If an address in the `holders` array increases their Option token balance after the batch is built but before it executes on-chain, the batch will unknowingly attempt to exercise the holder's larger live balance.

This creates an unbounded execution and batch-liveness risk for keepers, aggregators, or settlement contracts that construct batches off-chain based on expected holder balances. An authorized holder can grief a batch by increasing their option balance until the caller lacks the sufficient consideration token balance, causing the entire batch to revert. If the caller has sufficient funds and broad approvals, the caller can instead be forced to exercise more than intended, potentially exceeding inventory limits, risk budget, profitability assumptions, or off-chain settlement obligations.

Recommendation: Add caller-controlled bounds to the batch exercise path to prevent unintentional overspends. For example, replace the address-only batch with entries containing both the holder and the maximum amount the caller is willing to exercise:

GRK-2 Approved Operators Have Significant Spending Power

Low Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > This is a fair presentation but really should live as informational, the same way that calling approve() on erc20 is a risk. this is basically identical. in addition, there is an allowance baked into the factory as well that can be rescinded.

File(s) affected: `contracts/Option.sol`, `contracts/Factory.sol`

Description: The `Option.transferFrom()` function spends a standard per-option ERC-20 allowance unless the caller is already a factory-approved operator, then calls `_settledTransfer()`. `_settledTransfer()` does not distinguish between a direct transfer, an approved operator transfer, and a normal ERC-20 allowance transfer.

If `from` has enabled `Factory.enableAutoMintBurn(true)` and the requested transfer amount exceeds `from`'s current option balance, `_settledTransfer()` mints the deficit by calling `mint_(from, value - balance)`. That mint path calls `Receipt.mint()`, which pulls collateral from `from` through the factory allowance registry, so a spender approved only through `Option.approve(spender, amount)` can mint fresh options against the holder's collateral allowance by transferring more options than the holder currently owns. This means that the approved operator can handle the user's collateral token balance and Option tokens, thereby extending the approved operator's powers beyond expectations.

A malicious operator would be able to drain the collateral/consideration token allowances given to the factory by the user instantly without any chance for a normal user to first revoke their approval.

Given their heightened control over all tokens that the user has given the Factory contract a spend allowance for, users must completely trust any operators they choose to approve.

Recommendation: Consider preventing auto-mints from occuring within the `transferFrom()` function. One way to acheive this would be to first check that `balanceOf(from) >= amount` thus only allowing auto-mints to occur via a user controlled `transfer()` call.

GRK-3
Unbounded `strike` Can Overflow `Receipt.toConsideration()` and Block Settlement

Low Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `fdca46c8d979ac773db47277b71fc49b8527d178`.

File(s) affected: `foundry/contracts/Factory.sol`, `foundry/contracts/Receipt.sol`

Description: `Factory.createOption()` accepts any nonzero `strike` without checking whether `Receipt.toConsideration()` can safely multiply `strike() * numer()`. In markets where `consDecimals` is greater than `decimals`, `numer()` can be greater than `1`, so an unsafe `strike` can pass option creation and minting but later panic during exercise or redemption.

```
function toConsideration(uint256 amount, bool round) public pure returns (uint256) {
    if (!round) return Math.mulDiv(amount, strike() * numer(), (10 ** STRIKEDEC) * denom());
    return Math.mulDiv(amount, strike() * numer(), (10 ** STRIKEDEC) * denom(), Math.Rounding.Ceil);
}
```

A malicious or mistaken market creator can deploy an option whose settlement paths revert once `Receipt.toConsideration()` is reached. `Receipt._redeem()` also calls `Receipt.toConsideration()` before post-window collateral-only redemption, so affected receipt holders can be unable to redeem collateral, leaving receipt supply outstanding and blocking residual sweeps.

Recommendation: Reject unsafe strikes in `Factory.createOption()` using the same decimal scaling used by `Receipt.numer()`, for example by requiring `p.strike <= type(uint256).max / numerFor(collDec, consDec)`. As defense in depth, update `Receipt._redeem()` to skip `Receipt.toConsideration()` when the consideration-backed amount is zero.

GRK-4
Non-Standard Erc-20 Behaviours Such as Fee-on-Transfer and Rebasing Tokens Are Not Supported by the Protocol

Low Fixed

Update
Marked as "Fixed" by the client.
Addressed in: `6fcc6a64bab73228a529a7ee3134ca21b3759180`.

File(s) affected: `foundry/contracts/Factory.sol`, `foundry/contracts/Receipt.sol`, `foundry/contracts/Option.sol`

Description: The protocol assumes ERC-20 transfers are exact and balance-preserving, but it does not account for non-standard token mechanics such as fee-on-transfer deductions or rebasing balance changes. As a result, internal accounting may diverge from actual token balances when such assets are used, potentially causing incorrect deposits, withdrawals, share calculations, or solvency checks. It should be clear that Options accounting will not operate correctly if the `collateral` or `consideration` tokens used implement any of these behaviours.

Recommendation: Ensure that this expectation is clearly documented within the codebase as well as on the user interface when users create new options.

GRK-5
Dust-Sized `Receipt._redeem()` Calls Can Burn Receipts for Zero Payout

Low Acknowledged

Update
Marked as "Acknowledged" by the client.
The client provided the following explanation:

> While reverting seems like a good idea, I think it leads me to take care of other scenarios, such as round-down situations. These are the users responsibilities and usually are in the regime of dust situations which is not that large of a concern.

File(s) affected: `foundry/contracts/Receipt.sol`

Description: `Receipt._redeem()` can burn a positive consideration-backed receipt amount even when `Receipt.toConsideration()` floors the payout to `0`. If `amount > 0`, `consAmount == 0`, and `remaining == 0`, the function increments `redeemed`, burns the receipts, and skips both transfer branches. Splitting a consideration-backed position into dust-sized receipt balances can leave the rounded-down consideration in `Receipt` until all receipts are burned and the owner calls `Receipt.sweep()`.

The loss per redemption is bounded to less than one smallest consideration token unit. It also cannot be forced on an arbitrary holder without a normal receipt transfer or holder-approved `Receipt.redeemFor()` authority.

Recommendation: In `Receipt._redeem()`, reject positive consideration-backed redemptions that floor to zero when there is no collateral payout:

```
if (amount > 0 && consAmount == 0 && remaining == 0) revert ZeroValue();
```

Alternatively, enforce and document a minimum redeem size so dust is aggregated before redemption.

GRK-6

Lifetime Counters Can Saturate and Permanently Disable `Option.exercise()`

Low Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `1442d802b1ed31b639b7b5045a12891da977899e`.

File(s) affected: `foundry/contracts/Receipt.sol`, `foundry/contracts/Option.sol`

Description: `Receipt.exercise()` and `Receipt._redeem()` store lifetime totals in `exercised` and `redeemed`, so a max-sized or impractically repeated self-neutral exercise and redeem cycle can push both counters to `type(uint256).max`. Once `exercised` is saturated, any later positive exercise through `Option.exercise()` or `Option.exerciseFor()` reaches `Receipt.exercise()` and reverts on `exercised += amount`.

```
function exercise(address account, uint256 amount) public onlyOption nonReentrant {
    uint256 consAmount = toConsideration(amount, true);
    if (consAmount == 0) revert ZeroValue();
    factory.transferFrom(account, address(this), consAmount, address(consideration()));
    collateral().safeTransfer(account, amount);
    exercised += amount;
}
```

The affected option clone can still mint and transfer tokens, and receipt holders can recover collateral after the deadline, but future long-side exercise is permanently disabled. Practical triggering requires extreme ERC20 supplies and balances or an impractical number of repeated cycles, so likelihood is low.

Recommendation: Track the live consideration-backed receipt balance instead of lifetime totals. Increment the live balance in `Receipt.exercise()` and decrement it in `Receipt._redeem()` when consideration-backed receipts are redeemed. Alternatively, cap lifetime exercise volume before updating `exercised` so the counter cannot enter a state that prevents future exercise.

GRK-7

`Receipt.redeem()` Does Not Explicitly Guard Pre-Expiry European Redemption Attempts

Informational Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `be67fc9c78c43d4890c8aaae896ad9d392c93190`.

File(s) affected: `foundry/contracts/Receipt.sol`

Description: European options are intended to be exercisable only during the post-expiry exercise window, and the long-side path enforces this through `Option.canExercise()`. Short-side redemption through `Receipt.redeem()` does not include an explicit branch for `isEuro() && block.timestamp < expirationDate()`.

In the current implementation, pre-expiry European redemption attempts are still prevented indirectly because European options cannot normally be exercised before expiry, there is no consideration-backed redemption pool, and the collateral leg remains unavailable until after

`exerciseDeadline`. The call therefore reverts with `ExerciseWindowOpen`, and this does not appear to permit early withdrawal of collateral.

The impact is limited to clarity and integration ergonomics. Because the revert reason is produced indirectly from pool state rather than from the European exercise schedule itself, frontends, SDKs, and indexers may handle the lifecycle less precisely.

Recommendation: Add an explicit pre-window check in the receipt redemption path for European options. For example, if `isEuro()` is true and `block.timestamp < expirationDate()`, revert with a dedicated `BeforeExerciseWindow` error.

GRK-8

`sweep()` Remains Unavailable While Any Receipt Dust Remains Outstanding

Informational | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > It is 100% ok if this function cannot be called. it was placed for any event that there may be locked dust or donations to the tokens.

File(s) affected: `foundry/contracts/Receipt.sol`

Description: `Receipt.sweep()` allows the factory owner to recover residual token balances from a receipt contract only after all receipts have been burned. The function reverts whenever `totalSupply() != 0`, which protects active receipt holders from owner sweeps.

However, the same guard means the function can remain unavailable indefinitely if any amount of receipt dust remains outstanding. In a production environment, it may be difficult to guarantee that every receipt holder fully redeems or burns their position, especially after secondary transfers, lost keys, abandoned accounts, or tiny residual balances.

As a result, residual collateral, consideration, donated tokens, or stray ERC-20 balances may remain stuck in the receipt contract if total receipt supply never reaches zero.

Recommendation: If sweep functionality is not required, consider removing it to reduce operational complexity. If it is required, document the limitation clearly and consider whether a dust-tolerant or time-delayed recovery mechanism is appropriate for non-core stray assets.

GRK-9

Late `Receipt.exercise()` State Update Breaks Checks-Effects-Interactions Ordering

Informational | Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `35a558a08516ab33a37ce4f436d031b60bf17789`.

File(s) affected: `foundry/contracts/Receipt.sol`

Description: `Receipt.exercise()` transfers collateral before incrementing `exercised`, leaving the exercise counter stale during the external token transfer. The current `nonReentrant` guard prevents reentrant `Receipt.redeem()`, `Receipt.redeem(uint256)`, and `Receipt.redeemFor()` calls from observing that stale value, so this is not exploitable in the current code.

The ordering is still fragile. A future path that reads `exercised` outside the same guard could rely on stale state during the token callback.

Recommendation: Move `exercised += amount` before `collateral().safeTransfer(account, amount)` in `Receipt.exercise()`. The update does not depend on the transfer result, and the reorder restores checks-effects-interactions ordering without changing intended exercise behavior.

GRK-10

Direct `OptionUtils.details()` Calls Can Return the Wrong Option Address

Informational | Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `2a6435dbe261a1110246aad4164bb8f52f485c98`.

File(s) affected: `foundry/contracts/OptionUtils.sol`, `foundry/contracts/Option.sol`, `foundry/contracts/Receipt.sol`

Description: `OptionUtils.details()` sets `OptionInfo.option` to `address(this)`, which is only correct when called through `Option.details()`. Direct calls to the public helper run in the helper or caller context, so the returned `option` field can point to that address instead of the real option clone.

No value-moving protocol path consumes this field. The risk is limited to integrations that call `OptionUtils.details()` directly and then route, display, or index the wrong option address.

Recommendation: Set `OptionInfo.option` from `Receipt.option()` inside `OptionUtils.details()`:

```
option: receipt.option(),
```

Keep `Option.details()` as the preferred integration entrypoint for option metadata.

Auditor Suggestions

S1 Enforce Uniqueness for Economically Identical Options Created Via `Factory`

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `e681b4723b8752899d1bf8d5aa5e71fa951a0b77`.

File(s) affected: `foundry/contracts/Factory.sol`

Description: `Factory.createOption()` validates the supplied parameters and then unconditionally deploys fresh `Option` and `Receipt` clones. The factory does not maintain a registry keyed by option parameters, does not deploy deterministic clones for each parameter set, and does not check whether an equivalent option already exists.

As a result, multiple option markets can be created with identical collateral, consideration, expiration, strike, put/call flag, European/American flag, and exercise window.

This does not directly break collateral accounting because each option pair has its own receipt contract and isolated collateral pool. The primary concern is market structure: liquidity, pricing, indexing, and UI assumptions can fragment across multiple markets that are economically equivalent.

Recommendation: Consider adding a canonical option registry keyed by the full option parameter set. `createOption()` could call an internal `_getOrCreateOption()` helper that returns an existing option when one has already been deployed, and only deploys a new pair when no canonical market exists.

S2 Remove Unused `afterExerciseWindow` Modifier

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `479a69b2f44be250fb7fb25ae8d56d8cca33f3a0`.

File(s) affected: `foundry/contracts/Receipt.sol`

Description: The `afterExerciseWindow` modifier is defined in `Receipt.sol` but is not used by any function in the current codebase.

Unused code increases maintenance overhead and may make future review more error-prone, particularly in lifecycle-sensitive contracts where timing gates are security-relevant.

Recommendation: Remove the unused modifier unless it is intentionally reserved for a near-term change.

S3 Refresh `lib/clone` Lock Metadata

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `d32be696f5362a5d9199c81328af869041801277`.

File(s) affected: `foundry/foundry.lock`, `foundry/lib/clone`

Description: `foundry/foundry.lock` records `lib/clone` as `v0.1.0` at `635e5d2e29e02f92684ab583ffe1b75fb45bd1da`, while the repository submodule pins `foundry/lib/clone` to `v0.2.0` at `ec7c0a8f8a96d1e1422782ceee8cb59f5bc7243a`. The compiled dependency follows the submodule pointer, so this is a provenance and reproducibility issue rather than an exploit path. The stale lock entry can mislead dependency review or future rebuilds that rely on lock metadata.

Recommendation: Regenerate or update `foundry/foundry.lock` so the `lib/clone` entry matches the pinned `v0.2.0` submodule commit. Keep dependency inventory and release documentation tied to `ec7c0a8f8a96d1e1422782ceee8cb59f5bc7243a`.

S4 Correct the `MAX_DATA_LENGTH` NatSpec Rationale

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `edc3d96a869656f72f64a7f308ec1a696c773573`.

File(s) affected: `src/ClonesWithImmutableArgs.sol`

Description: The NatSpec for `ClonesWithImmutableArgs.MAX_DATA_LENGTH` gives the wrong reason for the `24519` cap. The constant value is correct, but the comment says values above the cap overflow `uint16(runSize)` and can silently deploy a truncated proxy. `ClonesWithImmutableArgs.creation()` instead rejects oversized data with `DataTooLong()`, and `uint16(runSize)` does not truncate until much larger inputs.

Recommendation: Update the NatSpec to state that the cap comes from the EIP-170 runtime limit: `24576` bytes minus the `55` byte proxy runtime and the `2` byte length suffix. Explain that `ClonesWithImmutableArgs.creation()` reliably rejects `data.length > MAX_DATA_LENGTH` with `DataTooLong()`, and remove the claim about `uint16(runSize)` overflow or silent truncation near the cap.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/greekfi/clone`

- `c96...401 ./.gitignore`
- `487...66f ./.gitmodules`
- `fac...bd3 ./LICENSE`
- `963...78b ./README.md`
- `de4...3a7 ./foundry.lock`
- `eee...4eb ./foundry.toml`
- `8a8...019 ./src/Clone.sol`

- `faf...60d ./src/ClonesWithImmutableArgs.sol`
- `838...38a ./test/Clone.t.sol`
- `eda...464 ./test/SoladyEquivalence.t.sol`
- `979...434 ./test/UpstreamEquivalence.t.sol`

Repo: `https://github.com/greekfi/greekfi`

- `0dd...e34 ./.gitignore`
- `869...427 ./.gitmodules`
- `41f...a8a ./LICENSE`
- `4ed...02d ./README.md`
- `d83...d20 ./contracts/Factory.sol`
- `590...53c ./contracts/Option.sol`
- `7cc...7cc ./contracts/OptionUtils.sol`
- `f81...191 ./contracts/Receipt.sol`
- `e69...ab8 ./contracts/interfaces/IFactory.sol`
- `15d...e37 ./contracts/interfaces/IOption.sol`
- `4f4...403 ./contracts/interfaces/IReceipt.sol`
- `48a...b62 ./foundry.lock`
- `323...60b ./foundry.toml`
- `d0b...e70 ./remappings.txt`

Test Suite Results

Enter the following commands to run the protocol's test suite:

```
cd foundry
foundry install
foundry test
```

Producing the following output:

```
Ran 1 test for test/StrikeTest.t.sol:StrikeTest
[PASS] test_StrikeFormats() (gas: 86191)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 233.09ms (58.66ms CPU time)

Ran 31 tests for test/ExerciseWindow.t.sol:ExerciseWindowTest
[PASS] test_DeadlineEqualsExpirationPlusWindow() (gas: 24932)
[PASS] test_European_AfterWindow_Reverts() (gas: 333284)
[PASS] test_European_ExerciseOnBehalf_PreExpiry_Reverts() (gas: 328544)
[PASS] test_European_InWindow_Works() (gas: 405787)
[PASS] test_European_NamePrefixIsRCTE() (gas: 357378)
[PASS] test_European_PreExpiry_Reverts() (gas: 328979)
[PASS] test_European_ReportsFlag() (gas: 335469)
[PASS] test_ExerciseAfterWindow_Reverts() (gas: 25670)
[PASS] test_ExerciseExactlyAtDeadline_Allowed() (gas: 126361)
[PASS] test_ExerciseInWindow_Works() (gas: 125772)
[PASS] test_ExerciseOnBehalf_AfterWindow_Reverts() (gas: 25524)
[PASS] test_ExerciseOnBehalf_BatchSkipsBadEntries() (gas: 253575)
[PASS] test_ExerciseOnBehalf_BatchSkipsZeroBalance() (gas: 240806)
[PASS] test_ExerciseOnBehalf_InWindow_Works() (gas: 260106)
[PASS] test_ExerciseOnBehalf_OperatorAlone_Reverts() (gas: 144601)
[PASS] test_ExerciseOnBehalf_RevokeWorks() (gas: 125550)
[PASS] test_ExerciseOnBehalf_Unauthorised_Reverts() (gas: 118609)
[PASS] test_ExerciseOneSecondPastDeadline_Reverts() (gas: 25678)
[PASS] test_ExercisePreExpiry_Works() (gas: 120401)
[PASS] test_PairRedeem_AfterWindow_Reverts() (gas: 24082)
[PASS] test_PairRedeem_InWindow_Works() (gas: 75762)
[PASS] test_PairRedeem_PreExpiry_Works() (gas: 65646)
[PASS] test_RedeemAfterWindow_Works() (gas: 53244)
[PASS] test_RedeemDuringWindow_Reverts() (gas: 29617)
```

```
[PASS] test_RedeemPreExpiry_Reverts() (gas: 18134)
[PASS] test_SweepAfterWindow_Works() (gas: 48112)
[PASS] test_SweepBeforeWindow_Reverts() (gas: 19634)
[PASS] test_Transfer_ExactlyAtDeadline_Allowed() (gas: 60782)
[PASS] test_Transfer_InWindow_Works() (gas: 60386)
[PASS] test_Transfer_OneSecondPastDeadline_Reverts() (gas: 23972)
[PASS] test_Transfer_PreExpiry_Works() (gas: 54506)
Suite result: ok. 31 passed; 0 failed; 0 skipped; finished in 233.80ms (10.42ms CPU time)

Ran 12 tests for test/Reentrancy.t.sol:ReentrancyTest
[PASS] test_BurnReentersBurn_Rejected() (gas: 192930)
[PASS] test_BurnReentersRedeem_Rejected() (gas: 273356)
[PASS] test_ExerciseReentersBurn_Rejected() (gas: 245286)
[PASS] test_ExerciseReentersExercise_Rejected() (gas: 253546)
[PASS] test_MintReentersFactoryTransferFrom_Rejected() (gas: 270029)
[PASS] test_MintReentersOptionMint_Rejected() (gas: 201589)
[PASS] test_MintReentersOtherOptionMint_Rejected() (gas: 215710)
[PASS] test_MintReentersRedeem_Rejected() (gas: 280698)
[PASS] test_RedeemReentersBatchRedeem_Rejected() (gas: 208670)
[PASS] test_RedeemReentersRedeem_Rejected() (gas: 120157)
[PASS] test_StateConsistencyAfterExerciseReentry() (gas: 256229)
[PASS] test_StateConsistencyAfterMintReentry() (gas: 212720)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 234.03ms (8.12ms CPU time)

Ran 3 tests for test/InterfaceDriftGuard.t.sol:InterfaceDriftGuardTest
[PASS] test_exerciseEvent_pinFourParams() (gas: 305)
[PASS] test_iOption_signaturesResolve() (gas: 144931)
[PASS] test_iReceipt_signaturesResolve() (gas: 59971)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 233.59ms (8.85ms CPU time)

Ran 8 tests for test/Sweep.t.sol:SweepTest
[PASS] test_RedeemFor_DustHolderDoesNotBrick() (gas: 457676)
[PASS] test_RedeemProRata_DustConsiderationFloorsToZero() (gas: 448058)
[PASS] test_Sweep_AfterFullRedemption_CatchesDonation() (gas: 182502)
[PASS] test_Sweep_RecoversDonatedDustWhenSupplyZero() (gas: 120192)
[PASS] test_Sweep_RecoversStrayForeignToken() (gas: 518439)
[PASS] test_Sweep_RejectsNonOwner() (gas: 56896)
[PASS] test_Sweep_RejectsWhileReceiptsOutstanding() (gas: 172394)
[PASS] test_Sweep_RejectsZeroRecipient() (gas: 19844)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 233.45ms (10.60ms CPU time)

Ran 6 tests for test/Factory.t.sol:FactoryTest
[PASS] test_Create_AmericanWindowZero_IsLiteralZero() (gas: 190683)
[PASS] test_Create_CustomWindow() (gas: 188144)
[PASS] test_Create_EuropeanWindowZero_Reverts() (gas: 15192)
[PASS] test_PastExpiryReverts() (gas: 14797)
[PASS] test_SameTokenReverts() (gas: 14599)
[PASS] test_ZeroStrikeReverts() (gas: 14804)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 233.09ms (23.20ms CPU time)

Ran 60 tests for test/BranchCoverage.t.sol:BranchCoverageTest
[PASS] test_Factory_AllowExercise_Self_Reverts() (gas: 9669)
[PASS] test_Factory_AllowRedeem_Self_Reverts() (gas: 10153)
[PASS] test_Factory_AllowRedeem_Works() (gas: 34456)
[PASS] test_Factory_AllowRedeem_ZeroAddr_Reverts() (gas: 9275)
[PASS] test_Factory_ApproveOperator_Self_Reverts() (gas: 9163)
[PASS] test_Factory_Approve_NonZeroToken_Works() (gas: 16858)
[PASS] test_Factory_Approve_ZeroToken_Reverts() (gas: 8950)
[PASS] test_Factory_CreateOption_AmericanZeroWindow_Works() (gas: 190298)
[PASS] test_Factory_CreateOption_CollDecimalsTooHigh_Reverts() (gas: 461605)
[PASS] test_Factory_CreateOption_ConsDecimalsTooHigh_Reverts() (gas: 461838)
[PASS] test_Factory_CreateOption_EqualTokens_Reverts() (gas: 13112)
[PASS] test_Factory_CreateOption_EuroNonZeroWindow_Works() (gas: 188584)
[PASS] test_Factory_CreateOption_EuroZeroWindow_Reverts() (gas: 15416)
[PASS] test_Factory_CreateOption_PastExpiry_Reverts() (gas: 15419)
[PASS] test_Factory_CreateOption_ZeroCollateral_Reverts() (gas: 13706)
[PASS] test_Factory_CreateOption_ZeroConsideration_Reverts() (gas: 13279)
[PASS] test_Factory_CreateOption_ZeroStrike_Reverts() (gas: 15395)
[PASS] test_Factory_TransferFrom_FeeOnTransfer_Reverts() (gas: 768872)
[PASS] test_Factory_TransferFrom_InsufficientAllowance_Reverts() (gas: 38494)
[PASS] test_Factory_TransferFrom_Success() (gas: 80427)
[PASS] test_Factory_TransferFrom_UnregisteredCaller_Reverts() (gas: 13879)
```

```
[PASS] test_Option_AutoBurn_OnReceive() (gas: 173233)
[PASS] test_Option_AutoMint_OnTransfer() (gas: 154995)
[PASS] test_Option_Burn_AfterDeadline_Reverts() (gas: 24742)
[PASS] test_Option_Burn_Zero_Reverts() (gas: 12709)
[PASS] test_Option_ExerciseAll_NoArg_Works() (gas: 108377)
[PASS] test_Option_ExerciseForBatch_AfterDeadline_Reverts() (gas: 25251)
[PASS] test_Option_ExerciseForBatch_EuropeanPreExpiry_Reverts() (gas: 329468)
[PASS] test_Option_ExerciseForBatch_SkipsUnauthorized() (gas: 208447)
[PASS] test_Option_ExerciseForBatch_SkipsZeroBalance() (gas: 31452)
[PASS] test_Option_ExerciseFor_Authorized_Works() (gas: 189194)
[PASS] test_Option_ExerciseFor_ReturnsAmount() (gas: 119483)
[PASS] test_Option_ExerciseFor_Unauthorized_Reverts() (gas: 69058)
[PASS] test_Option_Exercise_AfterDeadline_Reverts() (gas: 25055)
[PASS] test_Option_Exercise_EuropeanInWindow_Works() (gas: 343151)
[PASS] test_Option_Exercise_EuropeanPreExpiry_Reverts() (gas: 329270)
[PASS] test_Option_Init_AlreadyInitialized_Reverts() (gas: 17019)
[PASS] test_Option_MintFor_Operator_Works() (gas: 144882)
[PASS] test_Option_MintFor_Unauthorized_Reverts() (gas: 22110)
[PASS] test_Option_Mint_AfterExpiry_Reverts() (gas: 23456)
[PASS] test_Option_Mint_Zero_Reverts() (gas: 20850)
[PASS] test_Option_TransferFrom_AfterDeadline_Reverts() (gas: 52397)
[PASS] test_Option_TransferFrom_Operator_SkipsAllowance() (gas: 90695)
[PASS] test_Option_TransferFrom_SpendsAllowance() (gas: 70079)
[PASS] test_Option_Transfer_AfterDeadline_Reverts() (gas: 26249)
[PASS] test_Option_Transfer_InWindow_Works() (gas: 56248)
[PASS] test_Option_Transfer_InsufficientBalance_NoOptIn_Reverts() (gas: 33077)
[PASS] test_Receipt_Burn_NotOption_Reverts() (gas: 13078)
[PASS] test_Receipt_Exercise_NotOption_Reverts() (gas: 11947)
[PASS] test_Receipt_Mint_NotOption_Reverts() (gas: 12825)
[PASS] test_Receipt_PutName_InvertsStrike() (gas: 204302)
[PASS] test_Receipt_RedeemFor_SkipsUnauthorized() (gas: 135674)
[PASS] test_Receipt_RedeemFor_SkipsZeroBalance() (gas: 214703)
[PASS] test_Receipt_Redeem_CollateralLeg_Works() (gas: 67262)
[PASS] test_Receipt_Redeem_CollateralPoolDrained_Reverts() (gas: 62867)
[PASS] test_Receipt_Redeem_ConsFirst_PreWindow_Works() (gas: 146349)
[PASS] test_Receipt_Redeem_ConsPoolDrained_Reverts() (gas: 181023)
[PASS] test_Receipt_Redeem_PartialConsThenCollateral() (gas: 180572)
[PASS] test_Receipt_Redeem_PreWindowEmptyPool_Reverts() (gas: 18640)
[PASS] test_Receipt_Redeem_Zero_Reverts() (gas: 12144)
Suite result: ok. 60 passed; 0 failed; 0 skipped; finished in 241.69ms (19.54ms CPU time)

Ran 2 tests for test/FeeOnTransfer.t.sol:FeeOnTransferTest
[PASS] test_FeeOnTransferFailsAtMint() (gas: 325166)
[PASS] test_NormalTokensPassBalanceCheck() (gas: 463028)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 775.46µs (238.33µs CPU time)

Ran 5 tests for test/ImmutableArgsLayout.t.sol:ImmutableArgsLayoutTest
[PASS] test_layout_AmericanCall_18_6() (gas: 1070634)
[PASS] test_layout_AmericanPut_zeroWindow() (gas: 1073074)
[PASS] test_layout_EuropeanCall_36_36() (gas: 1070470)
[PASS] test_layout_EuropeanPut_6_18() (gas: 1070710)
[PASS] test_layout_StrikeAtUint96Max() (gas: 1070621)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.05ms (642.00µs CPU time)

Ran 5 tests for test/InvariantsDeep.t.sol:ConversionMathFuzzTest
[PASS] testFuzz_CeilFloorBudget(uint256) (runs: 256, µ: 12612, ~: 12535)
[PASS] testFuzz_MonotonicConsideration(uint256,uint256) (runs: 256, µ: 17531, ~: 17580)
[PASS] testFuzz_RoundTripCeilBoundedBelow(uint256) (runs: 256, µ: 12542, ~: 12444)
[PASS] testFuzz_RoundTripFloorBoundedAbove(uint256) (runs: 256, µ: 12372, ~: 12288)
[PASS] test_IdentityZero() (gas: 14469)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 359.24ms (433.52ms CPU time)

Ran 35 tests for test/GasAnalysis.t.sol:GasAnalysis
[PASS] testFuzz_Gas_Factory_CreateMultipleOptions(uint8) (runs: 256, µ: 1315294, ~: 1153119)
[PASS] testFuzz_Gas_Option_Exercise(uint256,uint256) (runs: 256, µ: 235908, ~: 238155)
[PASS] testFuzz_Gas_Option_Mint(uint256) (runs: 256, µ: 159370, ~: 159271)
[PASS] testFuzz_Gas_Option_Redeem(uint256,uint256) (runs: 256, µ: 172208, ~: 174665)
[PASS] test_Gas_Collateral_Approve() (gas: 32639)
[PASS] test_Gas_Collateral_BalanceOf() (gas: 11180)
[PASS] test_Gas_Collateral_BalancesOf() (gas: 176737)
[PASS] test_Gas_Collateral_Exercise() (gas: 233723)
[PASS] test_Gas_Collateral_Mint() (gas: 106542)
```

```
[PASS] test_Gas_Collateral_RedeemConsLeg() (gas: 251022)
[PASS] test_Gas_Collateral_Redeem_PostExpiration() (gas: 180461)
[PASS] test_Gas_Collateral_Redeem_PreExpiration() (gas: 173958)
[PASS] test_Gas_Collateral_Sweep_MultipleUsers() (gas: 279344)
[PASS] test_Gas_Collateral_Sweep_SingleUser() (gas: 139580)
[PASS] test_Gas_Collateral_Transfer() (gas: 185954)
[PASS] test_Gas_Collateral_TransferFrom() (gas: 212597)
[PASS] test_Gas_Factory_CreateOption() (gas: 181495)
[PASS] test_Gas_Factory_CreateOption_DirectCall() (gas: 178976)
[PASS] test_Gas_Factory_CreateOptions_20() (gas: 3259361)
[PASS] test_Gas_Option_Approve() (gas: 32812)
[PASS] test_Gas_Option_BalanceOf() (gas: 10795)
[PASS] test_Gas_Option_BalancesOf() (gas: 176363)
[PASS] test_Gas_Option_Details() (gas: 58325)
[PASS] test_Gas_Option_MintToAddress() (gas: 237287)
[PASS] test_Gas_Option_ToCollateral() (gas: 9620)
[PASS] test_Gas_Option_ToConsideration() (gas: 9792)
[PASS] test_Gas_Option_Transfer() (gas: 189449)
[PASS] test_Gas_Option_TransferFrom() (gas: 214801)
[PASS] test_Gas_Option_TransferFrom_AutoRedeem() (gas: 230635)
[PASS] test_Gas_Option_Transfer_AutoMint() (gas: 195876)
[PASS] test_Gas_Swap_OperatorAutoMint() (gas: 225244)
[PASS] test_Gas_Swap_OperatorAutoMint_AllCold() (gas: 224892)
[PASS] test_Gas_Workflow_FullLifecycle() (gas: 297042)
[PASS] test_Gas_Workflow_MultipleUsers() (gas: 383861)
[PASS] test_Gas_Workflow_PostExpiration() (gas: 259859)
Suite result: ok. 35 passed; 0 failed; 0 skipped; finished in 431.89ms (412.87ms CPU time)

Ran 2 tests for test/InvariantsDeep.t.sol:ProRataLinearityFuzzTest
[PASS] testFuzz_EqualBalancesIdenticalPayout(uint256,uint256) (runs: 256, μ: 300992, ~: 306312)
[PASS] testFuzz_SplitRedeemEquivalence(uint256,uint256) (runs: 256, μ: 317084, ~: 317251)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 16.26s (365.89ms CPU time)

Ran 71 tests for test/Option.t.sol:OptionTest
[PASS] testFuzz_MintAndExercise(uint256) (runs: 256, μ: 199163, ~: 199113)
[PASS] testFuzz_MintAndRedeem(uint256) (runs: 256, μ: 143520, ~: 143463)
[PASS] testFuzz_TransferAutoRedeem(uint256,uint256) (runs: 256, μ: 199363, ~: 200383)
[PASS] test_ApproveOperator() (gas: 210560)
[PASS] test_ApproveOperatorCannotApproveSelf() (gas: 10323)
[PASS] test_BalancesOf() (gas: 177087)
[PASS] test_BatchSweepMultipleUsers() (gas: 283303)
[PASS] test_CreateOptionPastExpirationReverts() (gas: 15207)
[PASS] test_CreateOptionSameTokenReverts() (gas: 12441)
[PASS] test_CreateOptionZeroStrikeReverts() (gas: 15805)
[PASS] test_CreateOptionsBatch() (gas: 514168)
[PASS] test_DecimalConversionRoundtrip() (gas: 13189)
[PASS] test_Details() (gas: 65819)
[PASS] test_DirectShortTransfer() (gas: 189339)
[PASS] test_DoubleExercise() (gas: 191837)
[PASS] test_Exercise1() (gas: 198676)
[PASS] test_ExerciseAfterWindow() (gas: 166246)
[PASS] test_ExerciseAllThenRedeem() (gas: 190048)
[PASS] test_ExerciseDuringWindow() (gas: 189580)
[PASS] test_ExerciseEmitsEvent() (gas: 188836)
[PASS] test_ExerciseThenTransferCollateralThenRedeem() (gas: 285006)
[PASS] test_ExerciseWithInsufficientConsideration() (gas: 233003)
[PASS] test_FactoryAllowanceDecrement() (gas: 172888)
[PASS] test_FactoryAllowanceInfinite() (gas: 168352)
[PASS] test_FactoryAllowanceInsufficient() (gas: 40019)
[PASS] test_FactoryTransferFromNonCollateral() (gas: 14877)
[PASS] test_FullLifecycle1() (gas: 490013)
[PASS] test_FullLifecycle2() (gas: 227387)
[PASS] test_InsufficientBalanceExercise() (gas: 166023)
[PASS] test_InsufficientBalanceRedeem() (gas: 165018)
[PASS] test_Mint() (gas: 169270)
[PASS] test_MintAfterExpiration() (gas: 22233)
[PASS] test_MintEmitsEvent() (gas: 164027)
[PASS] test_MintToAddress() (gas: 246351)
[PASS] test_MixedDecimals_18_6() (gas: 698817)
[PASS] test_MixedDecimals_6_18() (gas: 828150)
[PASS] test_MixedDecimals_ExerciseFlow() (gas: 843781)
[PASS] test_MultipleExerciseSessions() (gas: 284828)
```

```
[PASS] test_MultipleRedeemSessions() (gas: 208765)
[PASS] test_MultipleUsers() (gas: 426593)
[PASS] test_OptionNameFormat() (gas: 69524)
[PASS] test_PairRedeemAfterDeadlineReverts() (gas: 165823)
[PASS] test_PostExpirationFlow() (gas: 191938)
[PASS] test_PutMintAndExercise() (gas: 363971)
[PASS] test_PutNameDisplay() (gas: 213906)
[PASS] test_PutPostExpirationRedeem() (gas: 375853)
[PASS] test_PutRedeem() (gas: 285572)
[PASS] test_ReceiptNameFormat() (gas: 44240)
[PASS] test_Redeem1() (gas: 144274)
[PASS] test_RedeemConsideration1() (gas: 223010)
[PASS] test_RedeemConsiderationInsufficientBalance() (gas: 197320)
[PASS] test_ReinitCloneReverts() (gas: 17086)
[PASS] test_ShortRedeemAfterExpiration() (gas: 145563)
[PASS] test_ShortRedeemMixedCollateralExactAmounts() (gas: 232851)
[PASS] test_ShortRedeemWithMixedCollateral() (gas: 222768)
[PASS] test_Strike2000_Exercise() (gas: 354450)
[PASS] test_Strike2000_ToCollateral() (gas: 186908)
[PASS] test_Strike2000_ToConsideration() (gas: 187614)
[PASS] test_Sweep() (gas: 146478)
[PASS] test_ToCollateral() (gas: 9837)
[PASS] test_ToConsideration() (gas: 10339)
[PASS] test_Transfer1() (gas: 206890)
[PASS] test_TransferAutoMint() (gas: 201264)
[PASS] test_TransferBothTokensToSameAddress() (gas: 201648)
[PASS] test_TransferChain() (gas: 275724)
[PASS] test_TransferFromAutoRedeem() (gas: 254511)
[PASS] test_TransferFromWithApproval() (gas: 224022)
[PASS] test_TransferTransfer() (gas: 249738)
[PASS] test_ZeroAmountExercise() (gas: 166773)
[PASS] test_ZeroAmountMint() (gas: 21642)
[PASS] test_ZeroAmountRedeem() (gas: 163676)
Suite result: ok. 71 passed; 0 failed; 0 skipped; finished in 18.16s (18.08s CPU time)

Ran 4 tests for test/Adversarial.t.sol:InvariantFuzzTest
[PASS] invariant_collateralCoversOptions() (runs: 256, calls: 128000, reverts: 0)

╭-----------------+----------------+-------+---------+----------╮
| Contract        | Selector       | Calls | Reverts | Discards |
+=============================================================+
| InvariantHandler | exercise       | 21429 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | mint           | 21241 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | pairBurn       | 21073 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | redeemConsLeg  | 21494 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | transferOption | 21354 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | warpForward    | 21409 | 0       | 0        |
╰-----------------+----------------+-------+---------+----------╯

[PASS] invariant_considerationBound() (runs: 256, calls: 128000, reverts: 0)

╭-----------------+----------------+-------+---------+----------╮
| Contract        | Selector       | Calls | Reverts | Discards |
+=============================================================+
| InvariantHandler | exercise       | 21343 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | mint           | 21425 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | pairBurn       | 21361 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | redeemConsLeg  | 21178 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | transferOption | 21348 | 0       | 0        |
|-----------------+----------------+-------+---------+----------|
| InvariantHandler | warpForward    | 21345 | 0       | 0        |
╰-----------------+----------------+-------+---------+----------╯
```

[PASS] invariant_receiptSupplyGeOptionSupply() (runs: 256, calls: 128000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
InvariantHandler	exercise	21339	0	0
InvariantHandler	mint	21091	0	0
InvariantHandler	pairBurn	21610	0	0
InvariantHandler	redeemConsLeg	21488	0	0
InvariantHandler	transferOption	21364	0	0
InvariantHandler	warpForward	21108	0	0

[PASS] invariant_supplyAccounting() (runs: 256, calls: 128000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
InvariantHandler	exercise	21042	0	0
InvariantHandler	mint	21363	0	0
InvariantHandler	pairBurn	21338	0	0
InvariantHandler	redeemConsLeg	21338	0	0
InvariantHandler	transferOption	21342	0	0
InvariantHandler	warpForward	21577	0	0

Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 19.38s (66.61s CPU time)

Ran 37 tests for test/Adversarial.t.sol:AdversarialTest
[PASS] testFuzz_Conservation_MintBurn(uint256) (runs: 256, μ: 106005, ~: 105945)
[PASS] testFuzz_Conservation_MintExerciseRedeem(uint256,uint256) (runs: 256, μ: 223621, ~: 226922)
[PASS] testFuzz_PartialExerciseThenBurn(uint256,uint256) (runs: 256, μ: 183716, ~: 183735)
[PASS] test_AC_ExercisorIsNotOperator() (gas: 61089)
[PASS] test_AC_FactoryTransferFromReject() (gas: 18679)
[PASS] test_AC_InitOnlyFromFactory() (gas: 16209)
[PASS] test_AC_NoSelfOperator() (gas: 12587)
[PASS] test_AC_OperatorIsNotExercisor() (gas: 60461)
[PASS] test_AC_ReceiptPrivilegedPathsReject() (gas: 24683)
[PASS] test_AC_ReinitClone() (gas: 15721)
[PASS] test_AC_TemplateInitReverts() (gas: 15289)
[PASS] test_AC_UnauthorizedExerciseForReverts() (gas: 33351)
[PASS] test_Econ_CrossOptionAllowanceBleed() (gas: 325754)
[PASS] test_Econ_DonationStranded() (gas: 59403)
[PASS] test_Econ_FullLifecycleNoLeftover() (gas: 141252)
[PASS] test_Econ_RedeemConsiderationFCFS() (gas: 186911)
[PASS] test_Econ_RedeemConsiderationUnderfunded() (gas: 18288)
[PASS] test_H1_NoForceMintIntoVictim() (gas: 22770)
[PASS] test_H1_OperatorCanMintForOwner() (gas: 110803)
[PASS] test_H2_AllowanceRevocable() (gas: 15670)
[PASS] test_H3_BatchSkipsZeroBalance() (gas: 167264)
[PASS] test_L4_NowExpiryReverts() (gas: 15309)
[PASS] test_M3_ExerciseFor_KeeperPaysKeeperGets() (gas: 173553)
[PASS] test_M4_RedeemForBatchSkipsEmpty() (gas: 96041)
[PASS] test_M5_PathologicalDecimalsRejected() (gas: 559381)
[PASS] test_OptIn_AutoBurnRedeemsCollateral() (gas: 123096)
[PASS] test_OptIn_AutoBurnRequiresReceiverOptIn() (gas: 91131)
[PASS] test_OptIn_AutoMintRequiresSenderOptIn() (gas: 30707)
[PASS] test_RedeemFor_UnauthorizedSkipped() (gas: 27255)
[PASS] test_Time_BurnAtExactDeadlineWorks() (gas: 75083)
[PASS] test_Time_BurnOneSecondPastDeadlineReverts() (gas: 24446)
[PASS] test_Time_ExerciseAtExactDeadlineWorks() (gas: 128501)
[PASS] test_Time_ExerciseOneSecondPastDeadlineReverts() (gas: 24780)

```
[PASS] test_Time_ReceiptTransferPastDeadlineOK() (gas: 56599)
[PASS] test_Time_RedeemAtExactDeadlineReverts() (gas: 32283)
[PASS] test_Time_RedeemOneSecondPastDeadlineOpens() (gas: 49929)
[PASS] test_Time_TransferPastDeadlineReverts() (gas: 26194)
Suite result: ok. 37 passed; 0 failed; 0 skipped; finished in 19.43s (19.28s CPU time)

Ran 8 tests for test/InvariantsDeep.t.sol:InvariantsDeepTest
[PASS] invariant_ExactCollateralAccounting() (runs: 256, calls: 128000, reverts: 0)

╭-------------+-----------------------+-------+---------+----------╮
| Contract    | Selector              | Calls | Reverts | Discards |
+=================================================================+
| DeepHandler | exercise              | 11611 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | mint                  | 11714 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | pairBurn              | 11627 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | redeemConsLeg         | 11741 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | redeemProRata         | 11518 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | toggleAutoMintBurn    | 11602 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | toggleExerciseAllowed | 11520 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | toggleOperator        | 11657 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | transferOption        | 11783 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | transferReceipt       | 11698 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | warpForward           | 11529 | 0       | 0        |
╰-------------+-----------------------+-------+---------+----------╯

[PASS] invariant_ExactConsiderationAccounting() (runs: 256, calls: 128000, reverts: 0)

╭-------------+-----------------------+-------+---------+----------╮
| Contract    | Selector              | Calls | Reverts | Discards |
+=================================================================+
| DeepHandler | exercise              | 11685 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | mint                  | 11447 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | pairBurn              | 11675 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | redeemConsLeg         | 11623 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | redeemProRata         | 11581 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | toggleAutoMintBurn    | 11690 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | toggleExerciseAllowed | 11602 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | toggleOperator        | 11730 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | transferOption        | 11608 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | transferReceipt       | 11588 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | warpForward           | 11771 | 0       | 0        |
╰-------------+-----------------------+-------+---------+----------╯

[PASS] invariant_FactoryHoldsNoUnderlying() (runs: 256, calls: 128000, reverts: 0)

╭-------------+-----------------------+-------+---------+----------╮
| Contract    | Selector              | Calls | Reverts | Discards |
+=================================================================+
| DeepHandler | exercise              | 11857 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
| DeepHandler | mint                  | 11635 | 0       | 0        |
|-------------+-----------------------+-------+---------+----------|
```

DeepHandler	pairBurn	11556	0	0
DeepHandler	redeemConsLeg	11654	0	0
DeepHandler	redeemProRata	11633	0	0
DeepHandler	toggleAutoMintBurn	11522	0	0
DeepHandler	toggleExerciseAllowed	11652	0	0
DeepHandler	toggleOperator	11668	0	0
DeepHandler	transferOption	11595	0	0
DeepHandler	transferReceipt	11567	0	0
DeepHandler	warpForward	11661	0	0

[PASS] invariant_FullReceiptSideSolvency() (runs: 256, calls: 128000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
DeepHandler	exercise	11583	0	0
DeepHandler	mint	11806	0	0
DeepHandler	pairBurn	11685	0	0
DeepHandler	redeemConsLeg	11601	0	0
DeepHandler	redeemProRata	11557	0	0
DeepHandler	toggleAutoMintBurn	11507	0	0
DeepHandler	toggleExerciseAllowed	11546	0	0
DeepHandler	toggleOperator	11843	0	0
DeepHandler	transferOption	11865	0	0
DeepHandler	transferReceipt	11535	0	0
DeepHandler	warpForward	11472	0	0

[PASS] invariant_ProtocolHoldsNoProtocolTokens() (runs: 256, calls: 128000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
DeepHandler	exercise	11599	0	0
DeepHandler	mint	11837	0	0
DeepHandler	pairBurn	11531	0	0
DeepHandler	redeemConsLeg	11662	0	0
DeepHandler	redeemProRata	11619	0	0
DeepHandler	toggleAutoMintBurn	11785	0	0
DeepHandler	toggleExerciseAllowed	11593	0	0
DeepHandler	toggleOperator	11572	0	0
DeepHandler	transferOption	11829	0	0
DeepHandler	transferReceipt	11406	0	0

| DeepHandler | warpForward | 11567 | 0 | 0 |
└─────────────┴──────────────────────┴───────┴─────────┴──────────┘

[PASS] invariant_SumOfOptionBalances() (runs: 256, calls: 128000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
DeepHandler	exercise	11418	0	0
DeepHandler	mint	11718	0	0
DeepHandler	pairBurn	11656	0	0
DeepHandler	redeemConsLeg	11683	0	0
DeepHandler	redeemProRata	11473	0	0
DeepHandler	toggleAutoMintBurn	11629	0	0
DeepHandler	toggleExerciseAllowed	11782	0	0
DeepHandler	toggleOperator	11658	0	0
DeepHandler	transferOption	11557	0	0
DeepHandler	transferReceipt	11648	0	0
DeepHandler	warpForward	11778	0	0

[PASS] invariant_SumOfReceiptBalances() (runs: 256, calls: 128000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
DeepHandler	exercise	11614	0	0
DeepHandler	mint	11587	0	0
DeepHandler	pairBurn	11635	0	0
DeepHandler	redeemConsLeg	11484	0	0
DeepHandler	redeemProRata	11795	0	0
DeepHandler	toggleAutoMintBurn	11708	0	0
DeepHandler	toggleExerciseAllowed	11748	0	0
DeepHandler	toggleOperator	11545	0	0
DeepHandler	transferOption	11645	0	0
DeepHandler	transferReceipt	11613	0	0
DeepHandler	warpForward	11626	0	0

[PASS] invariant_UntouchedActorStaysZero() (runs: 256, calls: 128000, reverts: 0)

Contract	Selector	Calls	Reverts	Discards
DeepHandler	exercise	11572	0	0
DeepHandler	mint	11694	0	0
DeepHandler	pairBurn	11607	0	0
DeepHandler	redeemConsLeg	11637	0	0

```
| DeepHandler | redeemProRata          | 11636 | 0       | 0         |
|-------------+------------------------+-------+---------+-----------|
| DeepHandler | toggleAutoMintBurn     | 11642 | 0       | 0         |
|-------------+------------------------+-------+---------+-----------|
| DeepHandler | toggleExerciseAllowed | 11789 | 0       | 0         |
|-------------+------------------------+-------+---------+-----------|
| DeepHandler | toggleOperator         | 11608 | 0       | 0         |
|-------------+------------------------+-------+---------+-----------|
| DeepHandler | transferOption         | 11650 | 0       | 0         |
|-------------+------------------------+-------+---------+-----------|
| DeepHandler | transferReceipt        | 11526 | 0       | 0         |
|-------------+------------------------+-------+---------+-----------|
| DeepHandler | warpForward            | 11639 | 0       | 0         |
╰-------------+------------------------+-------+---------+-----------╯

Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 26.39s (154.53s CPU time)

Ran 16 test suites in 26.46s (102.06s CPU time): 290 tests passed, 0 failed, 0 skipped (290 total tests)
```

Code Coverage

The code coverage of the test suite can be obtained with the following commands:

```
cd foundry
forge install
forge coverage --no-match-coverage "script/|test/|contracts/mocks"
```

Producing the following output:

File	% Lines	% Statements	% Branches	% Funcs
contracts/Factory.sol	100.00% (61/61)	100.00% (73/73)	100.00% (14/14)	100.00% (15/15)
contracts/Option.sol	100.00% (95/95)	98.13% (105/107)	88.24% (15/17)	100.00% (30/30)
contracts/OptionUtils.sol	98.00% (49/50)	96.59% (85/88)	66.67% (4/6)	100.00% (6/6)
contracts/Receipt.sol	97.83% (90/92)	96.61% (114/118)	87.50% (14/16)	96.67% (29/30)
Total	98.99% (295/298)	97.67% (377/386)	88.68% (47/53)	98.77% (80/81)

Changelog

- 2026-06-18 - Initial report
- 2026-06-26 - Final report
- 2026-07-28 - Final report update

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

© 2026 – Quantstamp, Inc.